
Algorithm 1 Desynchronisation Recovery – Authentication Phase

Precondition: D holds stale $m_{\text{curr}} = m_{\text{old}}$; AS holds $(m_{\text{curr}}, m_{\text{old}}, PID_{\text{curr}}, PID_{\text{old}})$

- 1: $PID = H(ID_D \| m_{\text{curr}})$ $\triangleright$ stale m_{curr} gives $PID = PID_{\text{old}}$
 - 2: D sends (PID, Y_{AS-D}^H, ts_1) to AS
 - 3: **if** $PID == PID_{\text{curr}}$ **then**
 - 4: AS verifies using m_{curr} $\triangleright$ normal path
 - 5: **else if** $PID == PID_{\text{old}}$ **then**
 - 6: AS verifies using m_{old} $\triangleright$ desync detected
 - 7: **else**
 - 8: AS rejects D
 - 9: **end if**
-

Algorithm 2 Desynchronisation Recovery – Key Exchange Phase

Precondition: desync path from Algorithm 1; AS uses $m_{\text{old}}, PID_{\text{old}}$

- 1: $n_1 = \text{rand}()$
- 2: $m_{\text{new}} = H(n_1)$
- 3: $m_H = m_{\text{new}} \oplus H(Y_D^{H'} \| m_{\text{old}} \| R'_D \| ID_{AS} \| PID_{\text{old}} \| ts_2)$
- 4: AS sends (m_H, ts_2) to D
- 5: $m_{\text{new}} = m_H \oplus H(Y_D^H \| m_{\text{curr}} \| R_D \| ID_{AS} \| PID \| ts_2)$ $\triangleright D$ uses $m_{\text{curr}} = m_{\text{old}}$ — recovers correctly
- 6: $m_{\text{old}} = m_{\text{curr}}; m_{\text{curr}} = m_{\text{new}}; PID = H(ID_D \| m_{\text{curr}})$

Result: D and AS re-synchronised; PID rotated
